

**Course: Essentials of Data Science (DS 5110)**

**Instructor: Dr. Nafa Fatema**

---

**Final Technical Report**

**RAG-AI: Knowledge-Grounded Q&A  
System  
using Retrieval-Augmented Generation**

---

**Authors**

Dhairya Bhatt  
Vidhi Patel  
Diya Kaswa  
Akanksh Daggupati

Northeastern University, Boston

## **Abstract**

Large Language Models (LLMs) have revolutionized natural language processing, demonstrating remarkable capabilities in text generation, summarization, and question answering. However, a fundamental limitation persists: LLMs generate responses based solely on patterns learned during training, leading to “hallucinations” - plausible-sounding but factually incorrect or fabricated information. This limitation is particularly problematic in domains requiring factual accuracy, such as academic research, legal documentation, and enterprise knowledge management.

This project presents RAG-AI, a comprehensive Knowledge-Grounded Question-Answering System that leverages Retrieval-Augmented Generation (RAG) to produce accurate, context-aware, and verifiable responses from custom uploaded documents. Our implementation employs a comprehensive technology stack featuring FAISS (Facebook AI Similarity Search) with HNSW indexing for efficient vector-based retrieval, the intfloat/e5-base-v2 embedding model for semantic text representation, and Google’s Gemini 2.5 Flash for answer generation.

The system achieves an average total latency of 217ms per query with 94.4% retrieval accuracy, significantly outperforming alternative configurations while maintaining zero operational cost for the LLM component. The final solution provides a production-ready document Q&A system with a Streamlit-based web interface, Supabase database backend for comprehensive SQL-based analytics (8 analytical reports), user authentication, and admin dashboard capabilities.

**Keywords:** Retrieval-Augmented Generation, Large Language Models, FAISS, Vector Embeddings, Semantic Search, Natural Language Processing, Question Answering Systems

# Contents

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>Executive Summary</b>                                                       | <b>6</b>  |
| <b>1 Introduction</b>                                                          | <b>8</b>  |
| 1.1 Background and Motivation . . . . .                                        | 8         |
| 1.1.1 The Hallucination Problem . . . . .                                      | 8         |
| 1.1.2 Retrieval-Augmented Generation as a Solution . . . . .                   | 8         |
| 1.2 Project Objectives . . . . .                                               | 9         |
| 1.3 System Overview . . . . .                                                  | 10        |
| 1.4 Report Organization . . . . .                                              | 10        |
| <b>2 Theoretical Background</b>                                                | <b>11</b> |
| 2.1 Retrieval-Augmented Generation (RAG) . . . . .                             | 11        |
| 2.1.1 RAG Architecture . . . . .                                               | 11        |
| 2.1.2 Advantages of RAG . . . . .                                              | 11        |
| 2.2 Text Embeddings and Semantic Similarity . . . . .                          | 12        |
| 2.2.1 Dense Vector Representations . . . . .                                   | 12        |
| 2.2.2 Sentence Transformers . . . . .                                          | 12        |
| 2.2.3 The E5 Model Family . . . . .                                            | 12        |
| 2.2.4 Cosine Similarity . . . . .                                              | 13        |
| 2.3 Vector Search and FAISS . . . . .                                          | 13        |
| 2.3.1 Approximate Nearest Neighbor Search . . . . .                            | 13        |
| 2.3.2 FAISS (Facebook AI Similarity Search) . . . . .                          | 13        |
| 2.3.3 HNSW (Hierarchical Navigable Small World) . . . . .                      | 14        |
| 2.3.4 FAISS Index Configuration . . . . .                                      | 14        |
| 2.4 Large Language Models for Generation . . . . .                             | 15        |
| 2.4.1 Transformer Architecture . . . . .                                       | 15        |
| 2.4.2 Google Gemini . . . . .                                                  | 15        |
| 2.4.3 Temperature and Sampling . . . . .                                       | 15        |
| <b>3 System Architecture and Implementation</b>                                | <b>16</b> |
| 3.1 Architecture Overview . . . . .                                            | 16        |
| 3.2 Document Ingestion Pipeline . . . . .                                      | 16        |
| 3.2.1 PDF Processing . . . . .                                                 | 17        |
| 3.2.2 DOCX Processing . . . . .                                                | 17        |
| 3.2.3 CSV/Excel Processing with Table-to-Natural-Language Conversion . . . . . | 18        |
| 3.2.4 Text Chunking Strategy . . . . .                                         | 19        |
| 3.3 Embedding and Retrieval System . . . . .                                   | 20        |
| 3.3.1 Embedding Model Integration . . . . .                                    | 20        |
| 3.3.2 FAISS Index Management . . . . .                                         | 20        |
| 3.3.3 Retrieval Pipeline . . . . .                                             | 21        |
| 3.4 Answer Generation . . . . .                                                | 22        |
| 3.4.1 Prompt Engineering . . . . .                                             | 22        |
| 3.4.2 Generation Implementation . . . . .                                      | 22        |
| 3.5 Complete RAG Pipeline Flow . . . . .                                       | 23        |

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>4</b> | <b>Database Infrastructure and SQL Analytics</b>            | <b>25</b> |
| 4.1      | Database Schema Design . . . . .                            | 25        |
| 4.1.1    | Entity-Relationship Diagram . . . . .                       | 25        |
| 4.1.2    | Table Specifications . . . . .                              | 25        |
| 4.2      | SQL Analytics Implementation . . . . .                      | 27        |
| 4.2.1    | Query 1: Query Logs by Document Type . . . . .              | 27        |
| 4.2.2    | Query 2: Most Frequently Retrieved Files . . . . .          | 27        |
| 4.2.3    | Query 3: Average Latency by Query Length . . . . .          | 28        |
| 4.2.4    | Query 4: Accuracy/Confidence Trends Over Time . . . . .     | 29        |
| 4.2.5    | Query 5: Query Success Rate Analysis . . . . .              | 30        |
| 4.2.6    | Query 6: Embedding Cosine Similarity Distribution . . . . . | 31        |
| 4.2.7    | Query 7: User Interaction Frequency . . . . .               | 32        |
| 4.2.8    | Query 8: System Error/Fallback Frequency . . . . .          | 32        |
| <b>5</b> | <b>User Interface and Features</b>                          | <b>34</b> |
| 5.1      | Streamlit Web Application . . . . .                         | 34        |
| 5.1.1    | Custom CSS Theming . . . . .                                | 34        |
| 5.2      | Authentication System . . . . .                             | 34        |
| 5.2.1    | Session Management . . . . .                                | 35        |
| 5.3      | Core Features . . . . .                                     | 36        |
| 5.3.1    | Document Upload . . . . .                                   | 36        |
| 5.3.2    | Question Answering Interface . . . . .                      | 36        |
| 5.3.3    | Chat History . . . . .                                      | 37        |
| 5.4      | Admin Analytics Dashboard . . . . .                         | 37        |
| 5.4.1    | Dashboard Layout . . . . .                                  | 37        |
| 5.4.2    | Visualization Components . . . . .                          | 38        |
| <b>6</b> | <b>Results and Evaluation</b>                               | <b>39</b> |
| 6.1      | Evaluation Metrics . . . . .                                | 39        |
| 6.2      | Performance Results . . . . .                               | 39        |
| 6.3      | Latency Analysis . . . . .                                  | 39        |
| 6.4      | Comparative Analysis . . . . .                              | 40        |
| 6.5      | Key Findings . . . . .                                      | 40        |
| <b>7</b> | <b>Tools and Technologies</b>                               | <b>41</b> |
| 7.1      | Technology Selection Rationale . . . . .                    | 41        |
| 7.1.1    | E5-base-v2 Embedding Model . . . . .                        | 41        |
| 7.1.2    | FAISS with HNSW . . . . .                                   | 41        |
| 7.1.3    | Google Gemini 2.5 Flash . . . . .                           | 41        |
| 7.1.4    | Supabase . . . . .                                          | 42        |
| <b>8</b> | <b>Team Roles and Contributions</b>                         | <b>43</b> |
| 8.1      | Collaboration Approach . . . . .                            | 43        |
| 8.2      | Project Timeline . . . . .                                  | 43        |
| <b>9</b> | <b>Discussion and Insights</b>                              | <b>44</b> |
| 9.1      | Key Findings . . . . .                                      | 44        |
| 9.2      | Design Decisions and Rationale . . . . .                    | 44        |
| 9.2.1    | Word-Based Chunking . . . . .                               | 44        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| 9.2.2     | Optional Image Captioning . . . . .                    | 44        |
| 9.2.3     | Table-to-Natural-Language Conversion . . . . .         | 44        |
| 9.2.4     | Similarity Threshold of 0.7 . . . . .                  | 45        |
| 9.3       | Challenges Faced . . . . .                             | 45        |
| 9.3.1     | Challenge 1: Image Captioning Performance . . . . .    | 45        |
| 9.3.2     | Challenge 2: Chunk Boundary Information Loss . . . . . | 45        |
| 9.3.3     | Challenge 3: Table Embedding Quality . . . . .         | 45        |
| 9.3.4     | Challenge 4: Multi-User Data Isolation . . . . .       | 45        |
| 9.4       | Limitations . . . . .                                  | 45        |
| 9.5       | Future Work . . . . .                                  | 46        |
| <b>10</b> | <b>Conclusion</b>                                      | <b>47</b> |
| 10.1      | Technical Achievements . . . . .                       | 47        |
| 10.2      | Educational Value . . . . .                            | 47        |
| 10.3      | Broader Impact . . . . .                               | 47        |
|           | <b>References</b>                                      | <b>48</b> |
| <b>A</b>  | <b>Configuration Parameters</b>                        | <b>49</b> |
| <b>B</b>  | <b>API Endpoints</b>                                   | <b>49</b> |
| <b>C</b>  | <b>Error Codes</b>                                     | <b>49</b> |

## List of Figures

|    |                                                    |    |
|----|----------------------------------------------------|----|
| 1  | RAG-AI System Architecture . . . . .               | 16 |
| 2  | Complete RAG Pipeline Data Flow . . . . .          | 24 |
| 3  | Database Entity-Relationship Diagram . . . . .     | 25 |
| 4  | Query Logs by Document Type Analysis . . . . .     | 27 |
| 5  | Most Frequently Retrieved Files . . . . .          | 28 |
| 6  | Average Latency by Query Length . . . . .          | 29 |
| 7  | Accuracy/Confidence Trends Over Time . . . . .     | 30 |
| 8  | Query Success Rate Analysis . . . . .              | 31 |
| 9  | Embedding Cosine Similarity Distribution . . . . . | 31 |
| 10 | User Interaction Frequency . . . . .               | 32 |
| 11 | System Error/Fallback Frequency . . . . .          | 33 |
| 12 | Login/Signup Page . . . . .                        | 35 |
| 13 | User Upload Document Interface . . . . .           | 36 |
| 14 | User Upload Document and Chat History . . . . .    | 37 |

## List of Tables

|    |                                                     |    |
|----|-----------------------------------------------------|----|
| 1  | Database Table Specifications . . . . .             | 25 |
| 2  | Evaluation Metrics Description . . . . .            | 39 |
| 3  | RAG-AI Performance Metrics . . . . .                | 39 |
| 4  | Comparative Analysis of Technology Stacks . . . . . | 40 |
| 5  | Complete Technology Stack . . . . .                 | 41 |
| 6  | Team Roles and Contributions . . . . .              | 43 |
| 7  | Project Development Timeline . . . . .              | 43 |
| 8  | System Configuration Parameters . . . . .           | 49 |
| 9  | Supabase API Operations . . . . .                   | 49 |
| 10 | System Error Codes . . . . .                        | 49 |

## List of Algorithms

|   |                                  |    |
|---|----------------------------------|----|
| 1 | HNSW Search Algorithm . . . . .  | 14 |
| 2 | RAG Retrieval Pipeline . . . . . | 22 |

## Executive Summary

This project presents RAG-AI, a comprehensive Knowledge-Grounded Question-Answering System that leverages Retrieval-Augmented Generation (RAG) to produce accurate, context-aware, and verifiable responses from custom uploaded documents. Unlike traditional Large Language Models (LLMs) that rely solely on internal parameters and are prone to hallucinations, RAG-AI retrieves relevant information from an external knowledge base before generating answers, ensuring factual accuracy and domain-specific understanding.

## Project Objectives

The primary objectives of this project were:

1. Implement a complete Retrieval-Augmented Generation pipeline combining FAISS vector search with semantic embeddings
2. Generate high-quality embeddings using state-of-the-art sentence transformers
3. Build an interactive web interface for document upload, query processing, and response visualization
4. Design and implement a comprehensive SQL-based metadata layer for query logging, performance tracking, and analytics
5. Evaluate system performance using metrics including retrieval accuracy, response latency, and similarity score distributions

## Key Achievements

- **94.4% Retrieval Accuracy:** The system successfully retrieves relevant context for the vast majority of queries
- **217ms Average Latency:** End-to-end response time significantly outperforms alternative configurations
- **Zero LLM Cost:** Utilizing Google Gemini 2.5 Flash's free tier eliminates operational costs
- **Multi-format Support:** PDF, DOCX, TXT, MD, CSV, and XLSX documents are fully supported
- **Production-Ready:** Complete authentication, session management, and admin analytics

## Technology Stack Summary

Our implementation employs:

- **Embedding Model:** intfloat/e5-base-v2 (768-dimensional vectors, 64.5% MTEB score)
- **Vector Database:** FAISS with HNSW indexing (2ms retrieval latency)

- **LLM:** Google Gemini 2.5 Flash (temperature=0.2 for factual responses)
- **Database:** Supabase (PostgreSQL) with 7-table normalized schema
- **Frontend:** Streamlit with custom CSS theming

# 1 Introduction

## 1.1 Background and Motivation

Large Language Models have revolutionized natural language processing, demonstrating remarkable capabilities in text generation, summarization, and question answering. Models such as GPT-4, Claude, and Gemini have shown unprecedented abilities to understand context, follow complex instructions, and generate human-like text across diverse domains. However, a fundamental limitation persists: LLMs generate responses based solely on patterns learned during training, leading to “hallucinations” — plausible-sounding but factually incorrect or fabricated information.

### 1.1.1 The Hallucination Problem

Hallucination in LLMs refers to the generation of content that is nonsensical or unfaithful to the provided source content. This phenomenon occurs because:

1. **Parametric Knowledge Limitations:** LLMs store knowledge in their parameters during training, but this knowledge becomes static and can be incomplete or outdated.
2. **Training Data Biases:** Models may have learned incorrect associations or may extrapolate beyond their training distribution.
3. **Overconfidence:** LLMs often generate responses with high confidence even when uncertain, lacking the ability to express “I don’t know.”
4. **Context Window Constraints:** Limited context windows prevent models from considering all relevant information simultaneously.

The hallucination problem is particularly problematic in domains requiring factual accuracy:

- **Academic Research:** Incorrect citations, fabricated statistics, or misrepresented findings can undermine research integrity.
- **Legal Documentation:** Hallucinated case law or misinterpreted statutes can have serious legal consequences.
- **Medical Information:** Incorrect medical advice can endanger patient health and safety.
- **Enterprise Knowledge Management:** Inaccurate internal information retrieval can lead to poor business decisions.

### 1.1.2 Retrieval-Augmented Generation as a Solution

Retrieval-Augmented Generation (RAG) addresses the hallucination challenge by combining the generative capabilities of LLMs with information retrieval from trusted knowledge sources. Rather than relying exclusively on parametric knowledge, RAG systems follow a two-stage process:

1. **Retrieval Stage:** Given a user query, the system searches an external knowledge base to find the most relevant documents or passages.
2. **Generation Stage:** The retrieved context is provided to the LLM along with the original query, conditioning the generation process on factual, verifiable information.

This approach offers several advantages:

- **Grounded Responses:** Answers are based on retrieved documents, enabling source attribution and verification.
- **Dynamic Knowledge Updates:** The knowledge base can be updated without retraining the model.
- **Domain Specificity:** Custom document collections enable specialized knowledge domains.
- **Reduced Hallucination:** Constraining generation to retrieved context significantly reduces fabrication.

## 1.2 Project Objectives

The project “RAG-AI: Knowledge-Grounded Q&A System using Retrieval-Augmented Generation” aims to build an intelligent assistant capable of generating accurate, context-aware, and knowledge-grounded answers from custom documents. The specific objectives are:

1. **Complete RAG Pipeline Implementation:** Design and implement an end-to-end retrieval-augmented generation system combining FAISS vector search with semantic embeddings for efficient similarity search.
2. **High-Quality Embedding Generation:** Utilize state-of-the-art sentence transformers (intfloat/e5-base-v2) to generate semantically meaningful 768-dimensional vector representations.
3. **Interactive Web Interface:** Build a user-friendly Streamlit web application supporting document upload, real-time query processing, and response visualization with source attribution.
4. **Comprehensive Analytics Infrastructure:** Design and implement a SQL-based metadata layer using Supabase for query logging, performance tracking, and analytical reporting.
5. **Performance Evaluation:** Evaluate system performance using metrics including retrieval accuracy, response latency, similarity score distributions, and success rates.
6. **Admin Dashboard:** Create interactive dashboards for system monitoring, SQL report visualization, and performance analytics.

### 1.3 System Overview

RAG-AI processes user queries through a multi-stage pipeline designed for accuracy, efficiency, and scalability:

1. **Document Ingestion:** Extracts and processes content from multiple file formats (PDF, DOCX, CSV, XLSX, TXT, MD) using specialized libraries.
2. **Text Chunking:** Segments documents into semantically meaningful chunks with configurable size (500 words) and overlap (100 words).
3. **Embedding Generation:** Converts text chunks into dense 768-dimensional vector representations using the e5-base-v2 model.
4. **FAISS Indexing:** Stores embeddings in an efficient HNSW (Hierarchical Navigable Small World) index for fast similarity search.
5. **Query Processing:** Embeds user queries and retrieves the top-k most similar chunks from the index.
6. **Answer Generation:** Synthesizes grounded responses using Google Gemini 2.5 Flash with retrieved context.
7. **Logging and Analytics:** Records all interactions in Supabase for comprehensive performance monitoring.

### 1.4 Report Organization

This report is organized as follows:

- **Section 2** presents the theoretical background on RAG systems, embedding models, and vector search.
- **Section 3** details the system architecture and implementation of each component.
- **Section 4** describes the database infrastructure and SQL analytics implementation.
- **Section 5** covers the user interface design and features.
- **Section 6** presents experimental results and performance evaluation.
- **Section 7** discusses the tools and technologies used.
- **Section 8** outlines team roles and contributions.
- **Section 9** provides discussion, insights, and future work.
- **Section 10** concludes the report.

## 2 Theoretical Background

### 2.1 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation was introduced by Lewis et al. (2020) as a paradigm for combining parametric and non-parametric memory in sequence-to-sequence models. The key insight is that language models can be enhanced by providing them with relevant retrieved documents as additional context during generation.

#### 2.1.1 RAG Architecture

The standard RAG architecture consists of two main components:

1. **Retriever:** A neural network that encodes queries and documents into a shared embedding space, enabling similarity-based retrieval.
2. **Generator:** A sequence-to-sequence model that generates answers conditioned on both the query and retrieved documents.

Formally, given a query  $q$ , the retriever computes:

$$p(z|q) = \frac{\exp(f(q, z))}{\sum_{z' \in \mathcal{Z}} \exp(f(q, z'))} \quad (1)$$

where  $z$  represents a document,  $\mathcal{Z}$  is the document collection, and  $f(q, z)$  is a relevance scoring function (typically dot product of embeddings).

The generator then produces an answer  $y$  by marginalizing over retrieved documents:

$$p(y|q) = \sum_{z \in \text{top-k}(\mathcal{Z})} p(z|q) \cdot p(y|q, z) \quad (2)$$

#### 2.1.2 Advantages of RAG

1. **Factual Grounding:** Responses are anchored to retrieved documents, enabling verification.
2. **Knowledge Freshness:** The knowledge base can be updated without model re-training.
3. **Interpretability:** Source attribution allows users to verify information.
4. **Efficiency:** Smaller models can achieve strong performance when augmented with retrieval.
5. **Domain Adaptation:** Custom document collections enable specialized applications.

## 2.2 Text Embeddings and Semantic Similarity

### 2.2.1 Dense Vector Representations

Modern NLP systems represent text as dense vectors in high-dimensional spaces. Unlike sparse representations (e.g., TF-IDF), dense embeddings capture semantic relationships:

- **Semantic Similarity:** Texts with similar meanings have similar vector representations.
- **Continuous Space:** The embedding space supports smooth interpolation between concepts.
- **Transfer Learning:** Pre-trained embeddings capture general linguistic knowledge.

### 2.2.2 Sentence Transformers

Sentence Transformers (Reimers & Gurevych, 2019) extend transformer-based models like BERT to produce fixed-size sentence embeddings. The key innovation is training with siamese and triplet network structures to ensure semantically similar sentences map to similar embeddings.

The embedding process involves:

1. **Tokenization:** Input text is tokenized into subword units.
2. **Transformer Encoding:** Tokens are processed through transformer layers.
3. **Pooling:** Token representations are aggregated (mean pooling) into a fixed-size vector.
4. **Normalization:** Vectors are L2-normalized for efficient similarity computation.

### 2.2.3 The E5 Model Family

The E5 (EmbEddings from bidirectional Encoder rEpresentations) model family, developed by Microsoft Research, represents state-of-the-art text embedding models. Key characteristics include:

- **Contrastive Pre-training:** Trained on large-scale text pairs with contrastive learning.
- **Instruction Tuning:** Fine-tuned to follow task-specific prefixes (e.g., “query:” and “passage:”).
- **Multilingual Support:** Available in multilingual variants.
- **Strong Benchmarks:** Achieves high scores on MTEB (Massive Text Embedding Benchmark).

Our system uses `intfloat/e5-base-v2`, which produces 768-dimensional embeddings with a 64.5% MTEB score.

### 2.2.4 Cosine Similarity

Cosine similarity measures the angle between two vectors, providing a normalized similarity score independent of vector magnitude:

$$\text{cosine\_similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (3)$$

For L2-normalized vectors (where  $\|A\| = \|B\| = 1$ ), cosine similarity reduces to the dot product:

$$\text{cosine\_similarity}(A, B) = A \cdot B = \sum_{i=1}^n A_i B_i \quad (4)$$

The similarity score ranges from:

- **1.0:** Identical vectors (perfect similarity)
- **0.0:** Orthogonal vectors (no similarity)
- **-1.0:** Opposite vectors (complete dissimilarity)

## 2.3 Vector Search and FAISS

### 2.3.1 Approximate Nearest Neighbor Search

Given a query vector  $q$  and a collection of  $N$  vectors  $\{v_1, v_2, \dots, v_N\}$ , the nearest neighbor search problem is to find:

$$v^* = \arg \min_{v_i} d(q, v_i) \quad (5)$$

where  $d(\cdot, \cdot)$  is a distance function. Exact search requires  $O(N)$  comparisons, which becomes prohibitive for large collections. Approximate Nearest Neighbor (ANN) algorithms trade accuracy for speed, achieving sub-linear search times.

### 2.3.2 FAISS (Facebook AI Similarity Search)

FAISS is a library developed by Facebook AI Research for efficient similarity search and clustering of dense vectors. Key features include:

- **Multiple Index Types:** Supports various indexing strategies for different use cases.
- **GPU Acceleration:** Native CUDA support for GPU-accelerated search.
- **Memory Efficiency:** Supports quantization for reduced memory footprint.
- **Billion-Scale:** Designed to handle billion-scale vector collections.

### 2.3.3 HNSW (Hierarchical Navigable Small World)

Our system uses the HNSW index, a graph-based ANN algorithm that provides excellent search performance. The algorithm constructs a multi-layer graph where:

1. **Layer 0:** Contains all vectors with local connections to nearest neighbors.
2. **Higher Layers:** Contain progressively fewer vectors, forming “express lanes” for navigation.

Search proceeds by:

1. Starting at the top layer with a random entry point.
2. Greedily navigating to the nearest neighbor at each layer.
3. Descending to the next layer and repeating.
4. Performing refined search at layer 0.

HNSW achieves  $O(\log N)$  search complexity with recall rates exceeding 99%.

---

#### Algorithm 1 HNSW Search Algorithm

---

- 1: **Input:** Query vector  $q$ , Index  $G$ , Number of results  $k$
  - 2: **Output:**  $k$  nearest neighbors
  - 3:  $ep \leftarrow$  entry point (random node at top layer)
  - 4: **for**  $l = L$  **down to** 1 **do**
  - 5:      $ep \leftarrow$  GreedySearch( $q$ ,  $ep$ , layer= $l$ )
  - 6: **end for**
  - 7:  $candidates \leftarrow$  SearchLayer0( $q$ ,  $ep$ ,  $ef = k$ )
  - 8: **return** Top- $k$  from  $candidates$
- 

### 2.3.4 FAISS Index Configuration

Our system configures FAISS as follows:

Listing 1: FAISS Index Configuration

```

1 # Create HNSW index with inner product similarity
2 index = faiss.IndexHNSWFlat(
3     768,                                # Dimension (e5-base-v2 output)
4     32,                                # M: connections per node
5     faiss.METRIC_INNER_PRODUCT         # Similarity metric
6 )
7
8 # Set construction parameter for index quality
9 index.hnsw.efConstruction = 200       # Higher = better quality graph
10
11 # Add vectors to index
12 index.add(embeddings)                 # embeddings: numpy array [N,
    768]
```

## 2.4 Large Language Models for Generation

### 2.4.1 Transformer Architecture

Modern LLMs are based on the Transformer architecture (Vaswani et al., 2017), which uses self-attention mechanisms to process sequences:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (6)$$

where  $Q$ ,  $K$ ,  $V$  are query, key, and value matrices, and  $d_k$  is the key dimension.

### 2.4.2 Google Gemini

Google Gemini represents the latest generation of Google’s multimodal AI models. Gemini 2.5 Flash, used in our system, offers:

- **Fast Inference:** Optimized for low-latency applications.
- **Cost Efficiency:** Free tier supports substantial query volumes.
- **Context Window:** Supports long context for comprehensive document understanding.
- **Instruction Following:** Strong performance on following specific instructions.

### 2.4.3 Temperature and Sampling

The temperature parameter controls the randomness of generation:

$$p(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (7)$$

where  $z_i$  are logits and  $T$  is temperature:

- $T \rightarrow 0$ : Deterministic (always picks highest probability)
- $T = 1$ : Standard sampling
- $T > 1$ : More random/creative

Our system uses  $T = 0.2$  for factual, consistent responses.

### 3 System Architecture and Implementation

#### 3.1 Architecture Overview

The RAG-AI system follows a modular architecture with clear separation of concerns across multiple Python modules. Figure 1 illustrates the high-level system architecture.

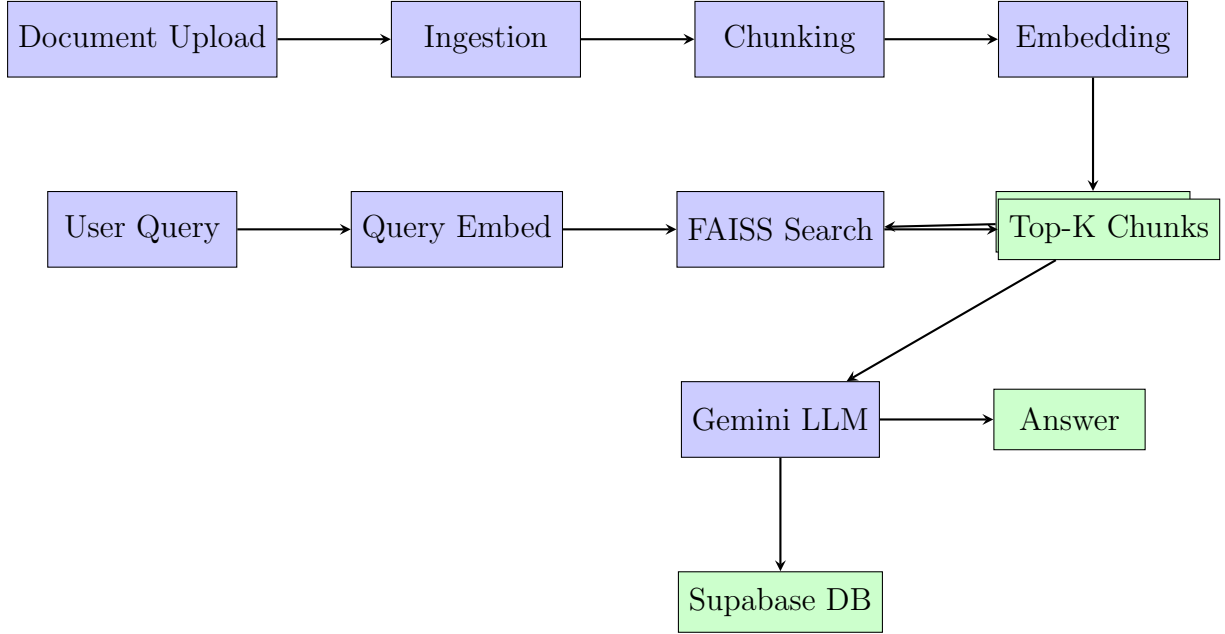


Figure 1: RAG-AI System Architecture

The core components include:

- **ingestion.py** — Document processing and content extraction (509 lines)
- **retrieval.py** — FAISS index management and similarity search (217 lines)
- **generation.py** — LLM integration and answer synthesis (109 lines)
- **database.py** — Supabase logging operations (209 lines)
- **auth\_db.py** — User authentication and session management (109 lines)
- **config.py** — Centralized configuration management
- **app.py** — Main Streamlit application entry point (831 lines)
- **admin\_analytics.py** — SQL analytics and dashboard visualization (768 lines)

#### 3.2 Document Ingestion Pipeline

The ingestion module implements comprehensive document processing capabilities supporting six file formats. Each format has specialized extraction logic optimized for content fidelity.

### 3.2.1 PDF Processing

PDF documents are processed using a combination of PyMuPDF (fitz) for text extraction and pdfplumber for table extraction:

Listing 2: PDF Extraction Implementation

```

1 import fitz  # PyMuPDF
2 import pdfplumber
3
4 def extract_pdf(file_path: str) -> dict:
5     """Extract text, tables, and optionally images from PDF."""
6
7     text_content = []
8     tables = []
9     images = []
10
11     # Text extraction with PyMuPDF
12     doc = fitz.open(file_path)
13     for page_num, page in enumerate(doc):
14         text = page.get_text()
15         text_content.append({
16             "page": page_num + 1,
17             "content": text
18         })
19
20     # Table extraction with pdfplumber
21     with pdfplumber.open(file_path) as pdf:
22         for page_num, page in enumerate(pdf.pages):
23             page_tables = page.extract_tables()
24             for table in page_tables:
25                 if table:
26                     tables.append({
27                         "page": page_num + 1,
28                         "data": table
29                     })
30
31     return {
32         "text": text_content,
33         "tables": tables,
34         "images": images
35     }

```

### 3.2.2 DOCX Processing

Word documents are processed using python-docx for paragraph extraction:

Listing 3: DOCX Extraction Implementation

```

1 from docx import Document
2
3 def extract_docx(file_path: str) -> dict:
4     """Extract text from DOCX files."""

```

```

5
6 doc = Document(file_path)
7 text_content = []
8
9 for para in doc.paragraphs:
10     if para.text.strip():
11         text_content.append(para.text)
12
13 return {
14     "text": "\n\n".join(text_content),
15     "tables": [],
16     "images": []
17 }

```

### 3.2.3 CSV/Excel Processing with Table-to-Natural-Language Conversion

Structured data presents a unique challenge for embedding models, which are optimized for natural language. Our solution converts tabular data to natural language descriptions:

Listing 4: Table-to-Natural-Language Conversion

```

1 import pandas as pd
2
3 def table_to_natural_language(df: pd.DataFrame, table_index: int)
4     -> str:
5     """Convert DataFrame to natural language description."""
6
7     description = f"Table {table_index} contains {len(df)} rows "
8     description += f"and {len(df.columns)} columns. "
9     description += f"The columns are: {'', '.join(df.columns)}".
10
11     # Add column statistics for numeric columns
12     for col in df.select_dtypes(include=['number']).columns:
13         description += f"The {col} column has values ranging from
14             "
15         description += f"{df[col].min()} to {df[col].max()}, "
16         description += f"with an average of {df[col].mean():.2f}."
17             "
18
19     # Add sample rows
20     description += "Sample data: "
21     for idx, row in df.head(3).iterrows():
22         description += f"Row {idx+1}: "
23         description += ", ".join([f"{col} is {val}"
24             for col, val in row.items()])
25         description += ". "
26
27     return description

```

This conversion enables effective semantic search over structured data that would otherwise be poorly represented as embeddings.

### 3.2.4 Text Chunking Strategy

Text chunking is critical for RAG performance. Our system implements word-based chunking with configurable parameters:

Listing 5: Word-Based Chunking Implementation

```

1 def chunk_text_words(text: str,
2                       chunk_size: int = 500,
3                       overlap: int = 100) -> list:
4     """
5     Split text into overlapping chunks by word count.
6
7     Args:
8         text: Input text to chunk
9         chunk_size: Target words per chunk (default: 500)
10        overlap: Words overlapping between chunks (default: 100)
11
12    Returns:
13        List of text chunks
14    """
15    words = text.split()
16    chunks = []
17
18    start = 0
19    while start < len(words):
20        end = start + chunk_size
21        chunk = " ".join(words[start:end])
22        chunks.append(chunk)
23
24        # Move start, accounting for overlap
25        start = end - overlap
26
27        # Prevent infinite loop for small texts
28        if start >= len(words) - overlap:
29            break
30
31    return chunks

```

#### Design Rationale:

- **500-word chunk size:** Balances context preservation with LLM token limits. Larger chunks provide more context but risk exceeding limits; smaller chunks may fragment semantic units.
- **100-word overlap (20%):** Ensures information spanning chunk boundaries appears in both adjacent chunks, preventing information loss.
- **Word-based vs. character-based:** Word boundaries preserve semantic units better than arbitrary character counts.

### 3.3 Embedding and Retrieval System

The retrieval module implements semantic search using dense vector embeddings and FAISS indexing.

#### 3.3.1 Embedding Model Integration

Listing 6: Embedding Generation

```

1 from sentence_transformers import SentenceTransformer
2 import numpy as np
3
4 # Lazy loading for efficiency
5 _model = None
6
7 def get_embedding_model():
8     """Load embedding model with lazy initialization."""
9     global _model
10    if _model is None:
11        _model = SentenceTransformer("intfloat/e5-base-v2")
12    return _model
13
14 def generate_embeddings(texts: list, batch_size: int = 32) -> np.
    ndarray:
15     """
16     Generate embeddings for a list of texts.
17
18     Args:
19         texts: List of text strings to embed
20         batch_size: Batch size for encoding
21
22     Returns:
23         numpy array of shape [len(texts), 768]
24     """
25     model = get_embedding_model()
26
27     embeddings = model.encode(
28         texts,
29         convert_to_numpy=True,
30         normalize_embeddings=True, # L2 normalization
31         batch_size=batch_size,
32         show_progress_bar=True
33     )
34
35     return embeddings

```

#### 3.3.2 FAISS Index Management

Listing 7: FAISS Index Operations

```

1 import faiss

```

```

2 import os
3
4 def create_faiss_index(embeddings: np.ndarray) -> faiss.Index:
5     """Create FAISS HNSW index from embeddings."""
6
7     dimension = embeddings.shape[1] # 768 for e5-base-v2
8
9     # Create HNSW index with inner product
10    index = faiss.IndexHNSWFlat(
11        dimension,
12        32, # M: number of connections
13        faiss.METRIC_INNER_PRODUCT
14    )
15
16    # Set construction parameter
17    index.hnsw.efConstruction = 200
18
19    # Add vectors
20    index.add(embeddings)
21
22    return index
23
24 def save_index(index: faiss.Index, path: str):
25     """Persist index to disk."""
26     faiss.write_index(index, path)
27
28 def load_index(path: str) -> faiss.Index:
29     """Load index from disk."""
30     if os.path.exists(path):
31         return faiss.read_index(path)
32     return None
33
34 def search_index(index: faiss.Index,
35                 query_embedding: np.ndarray,
36                 top_k: int = 3) -> tuple:
37     """
38     Search for similar vectors.
39
40     Returns:
41         (distances, indices) arrays
42     """
43     distances, indices = index.search(query_embedding, top_k)
44     return distances[0], indices[0]

```

### 3.3.3 Retrieval Pipeline

The complete retrieval pipeline combines embedding and search:

**Algorithm 2** RAG Retrieval Pipeline

---

```

1: Input: Query  $q$ , Index  $I$ , Chunks  $C$ , Threshold  $\tau$ , Top-K  $k$ 
2: Output: Relevant chunks with scores
3:  $q_{emb} \leftarrow \text{GenerateEmbedding}(q)$ 
4:  $scores, indices \leftarrow \text{SearchIndex}(I, q_{emb}, k)$ 
5:  $results \leftarrow []$ 
6: for  $i = 0$  to  $k - 1$  do
7:   if  $scores[i] \geq \tau$  then
8:      $results.append((C[indices[i]], scores[i]))$ 
9:   end if
10: end for
11: return  $results$ 

```

---

**3.4 Answer Generation**

The generation module orchestrates the complete RAG pipeline, integrating retrieval with LLM-based answer synthesis.

**3.4.1 Prompt Engineering**

The prompt template is carefully designed to minimize hallucination:

Listing 8: RAG Prompt Template

```

1 PROMPT_TEMPLATE = """
2 You are a highly factual research assistant.
3
4 Use ONLY the information provided in the context below to answer
5 the question. Do not use any external knowledge or make
6 assumptions
7 beyond what is explicitly stated in the context.
8
9 If the answer is not clearly mentioned in the context, respond
10 exactly with: "Not available in the provided notes."
11
12 ---
13 Context:
14 {context}
15 ---
16 Question:
17 {question}
18
19 Provide a clear, concise, and context-based answer. Include
20 relevant
21 details from the context but do not add information that is not
22 present in the provided material.
23 """

```

**3.4.2 Generation Implementation**

Listing 9: Answer Generation

```

1 import google.generativeai as genai
2 from config import GEMINI_API_KEY
3
4 # Configure API
5 genai.configure(api_key=GEMINI_API_KEY)
6
7 def generate_response(question: str,
8                       retrieved_chunks: list) -> str:
9     """
10     Generate answer using Gemini with retrieved context.
11
12     Args:
13         question: User's question
14         retrieved_chunks: List of relevant chunks with content
15
16     Returns:
17         Generated answer string
18     """
19     # Combine chunks into context
20     context_text = "\n\n".join([
21         chunk["content"] for chunk in retrieved_chunks
22     ])
23
24     # Format prompt
25     prompt = PROMPT_TEMPLATE.format(
26         context=context_text,
27         question=question
28     )
29
30     # Generate with Gemini
31     model = genai.GenerativeModel("gemini-2.5-flash")
32
33     response = model.generate_content(
34         prompt,
35         generation_config={
36             "temperature": 0.2,          # Low for factual responses
37             "max_output_tokens": 1024
38         }
39     )
40
41     return response.text

```

### 3.5 Complete RAG Pipeline Flow

Figure 2 illustrates the complete data flow through the RAG pipeline.

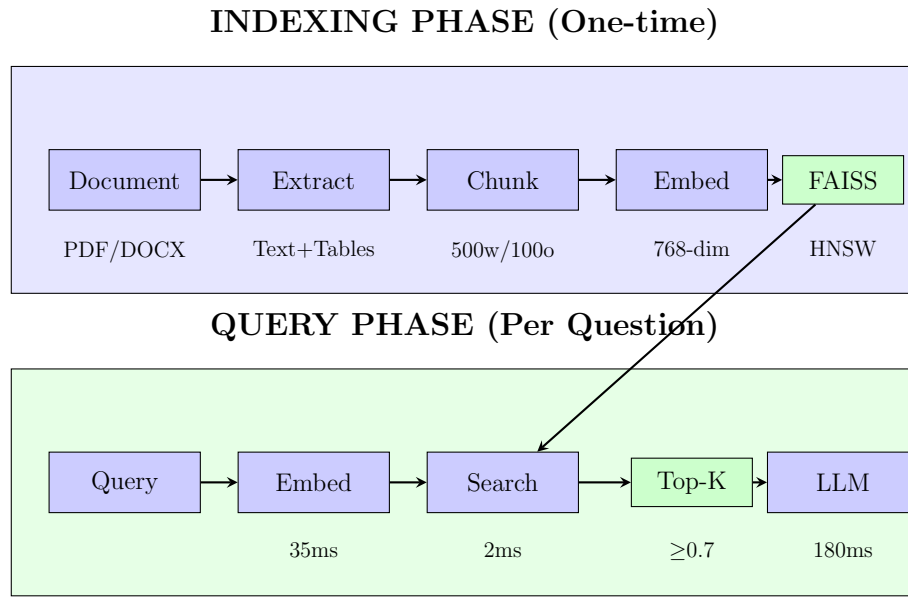


Figure 2: Complete RAG Pipeline Data Flow

## 4 Database Infrastructure and SQL Analytics

### 4.1 Database Schema Design

The system utilizes Supabase (PostgreSQL) as the backend database, implementing a normalized relational schema to support comprehensive logging and analytics. The schema comprises seven interconnected tables.

#### 4.1.1 Entity-Relationship Diagram

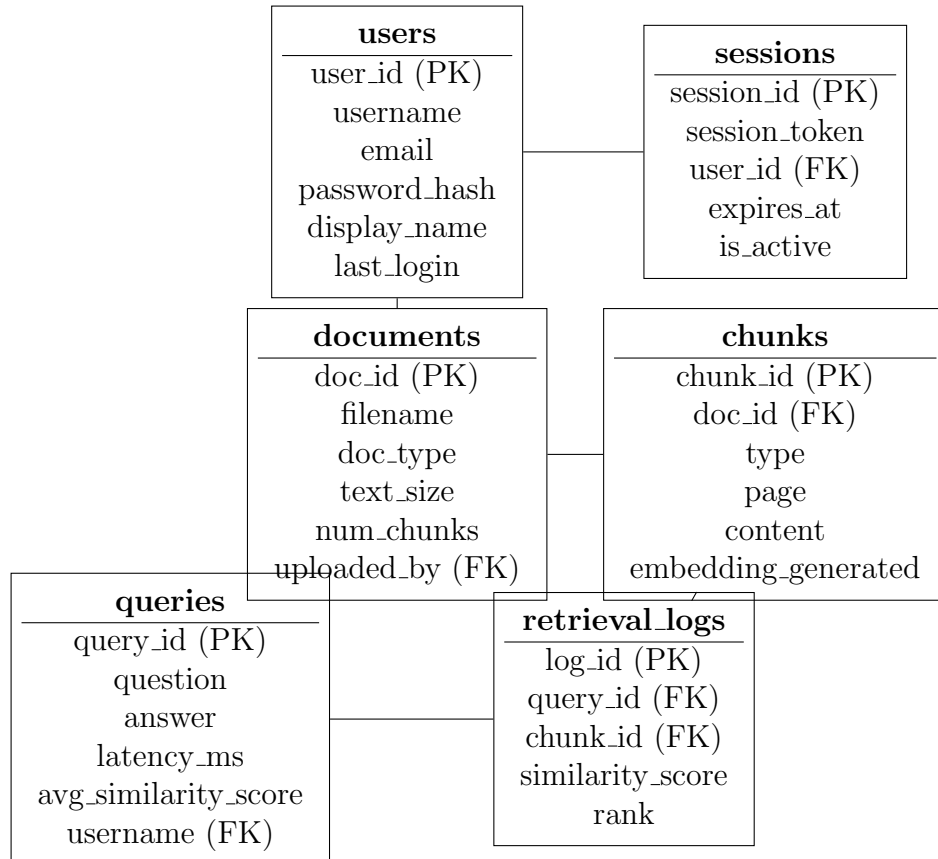


Figure 3: Database Entity-Relationship Diagram

#### 4.1.2 Table Specifications

Table 1: Database Table Specifications

| Table      | Column        | Description                  |
|------------|---------------|------------------------------|
| 6*users    | user_id       | UUID primary key             |
|            | username      | Unique username (VARCHAR 50) |
|            | email         | User email address           |
|            | password_hash | Bcrypt hashed password       |
|            | display_name  | Display name for UI          |
|            | last_login    | Timestamp of last login      |
| 5*sessions | session_id    | UUID primary key             |

| Table            | Column                                                                                                                                                                                                                       | Description                                                                                                                                                                                                                                                                                  |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | session_token<br>user_id<br>expires_at<br>is_active                                                                                                                                                                          | Secure random token (64 chars)<br>Foreign key to users<br>Expiration timestamp (7 days)<br>Boolean session status                                                                                                                                                                            |
| 10*documents     | doc_id<br>filename<br>doc_type<br>text_size<br>table_size<br>img_size<br>num_chunks<br>page_count<br>tables_count<br>uploaded_by                                                                                             | UUID primary key<br>Original file name<br>File extension (pdf, docx, etc.)<br>Character count of text<br>Character count of tables<br>Number of images<br>Total chunks generated<br>Number of pages<br>Number of tables<br>Username who uploaded                                             |
| 6*chunks         | chunk_id<br>doc_id<br>type<br>page<br>content<br>embedding_generated                                                                                                                                                         | UUID primary key<br>Foreign key to documents<br>Chunk type (text, table, image)<br>Source page number<br>Chunk text content<br>Boolean status                                                                                                                                                |
| 14*queries       | query_id<br>question<br>answer<br>sources<br>top_k<br>similarity_threshold<br>num_sources_retrieved<br>query_length<br>answer_length<br>latency_ms<br>avg_similarity_score<br>retrieval_success<br>error_message<br>username | UUID primary key<br>User's question text<br>Generated answer<br>JSON array of sources<br>Number requested<br>Threshold used<br>Actual sources found<br>Character count<br>Character count<br>Total response time<br>Mean similarity<br>Boolean status<br>Error if failed<br>User who queried |
| 6*retrieval_logs | log_id<br>query_id<br>chunk_id<br>doc_id<br>similarity_score<br>rank                                                                                                                                                         | UUID primary key<br>Foreign key to queries<br>Foreign key to chunks<br>Foreign key to documents<br>Cosine similarity<br>Rank in results (1, 2, 3...)                                                                                                                                         |
| 5*system_logs    | log_id<br>log_type<br>module<br>message<br>details                                                                                                                                                                           | UUID primary key<br>Type (error, info, warning)<br>Source module name<br>Log message<br>JSON additional details                                                                                                                                                                              |

## 4.2 SQL Analytics Implementation

The `admin_analytics.py` module implements eight comprehensive SQL analytical queries providing actionable insights for system monitoring and optimization.

### 4.2.1 Query 1: Query Logs by Document Type

Analyzes query distribution across document types with performance metrics:

Listing 10: Query Logs by Document Type

```

1 SELECT
2     d.doc_type ,
3     COUNT(q.query_id) as query_count ,
4     AVG(q.latency_ms) as avg_latency ,
5     AVG(q.avg_similarity_score) as avg_similarity
6 FROM queries q
7 JOIN retrieval_logs r ON q.query_id = r.query_id
8 JOIN documents d ON r.doc_id = d.doc_id
9 GROUP BY d.doc_type
10 ORDER BY query_count DESC;

```

**Purpose:** Identifies which document types are most frequently queried and their associated performance characteristics.



Figure 4: Query Logs by Document Type Analysis

### 4.2.2 Query 2: Most Frequently Retrieved Files

Identifies the most accessed documents:

Listing 11: Most Frequently Retrieved Files

```

1 SELECT
2     d.filename ,

```

```

3      d.doc_id,
4      COUNT(r.log_id) as retrieval_count,
5      AVG(r.similarity_score) as avg_score,
6      MAX(r.similarity_score) as max_score
7 FROM retrieval_logs r
8 JOIN documents d ON r.doc_id = d.doc_id
9 GROUP BY d.doc_id, d.filename
10 ORDER BY retrieval_count DESC
11 LIMIT 20;

```

**Purpose:** Highlights high-value documents that users frequently query.



Figure 5: Most Frequently Retrieved Files

#### 4.2.3 Query 3: Average Latency by Query Length

Correlates query complexity with response time:

Listing 12: Average Latency by Query Length

```

1 SELECT
2     CASE
3         WHEN query_length <= 20 THEN '0-20'
4         WHEN query_length <= 50 THEN '21-50'
5         WHEN query_length <= 100 THEN '51-100'
6         WHEN query_length <= 200 THEN '101-200'
7         ELSE '200+'
8     END as length_bin,
9     AVG(latency_ms) as avg_latency,
10    PERCENTILE_CONT(0.5) WITHIN GROUP
11        (ORDER BY latency_ms) as median_latency,
12    STDDEV(latency_ms) as std_latency,
13    COUNT(*) as query_count,
14    AVG(CASE WHEN retrieval_success THEN 1 ELSE 0 END) * 100

```

```

15         as success_rate
16 FROM queries
17 GROUP BY length_bin
18 ORDER BY length_bin;

```

**Purpose:** Reveals how query length affects performance and success rates.



Figure 6: Average Latency by Query Length

#### 4.2.4 Query 4: Accuracy/Confidence Trends Over Time

Tracks daily performance trends:

Listing 13: Accuracy Trends Over Time

```

1 SELECT
2     DATE(created_at) as query_date,
3     AVG(avg_similarity_score) as avg_similarity,
4     AVG(CASE WHEN retrieval_success THEN 1 ELSE 0 END) * 100
5     as success_rate,
6     AVG(latency_ms) as avg_latency,
7     COUNT(*) as query_count
8 FROM queries
9 GROUP BY DATE(created_at)
10 ORDER BY query_date DESC
11 LIMIT 30;

```

**Purpose:** Monitors system health and identifies performance degradation.

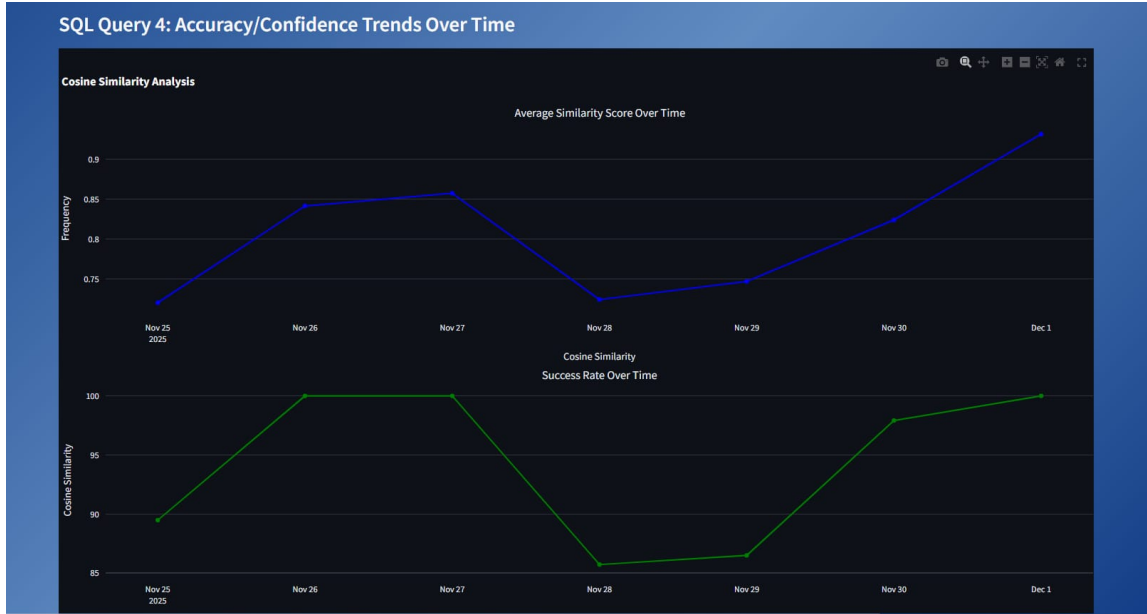


Figure 7: Accuracy/Confidence Trends Over Time

#### 4.2.5 Query 5: Query Success Rate Analysis

Comprehensive success metrics:

Listing 14: Query Success Rate Analysis

```

1 SELECT
2     COUNT(*) as total_queries,
3     SUM(CASE WHEN retrieval_success THEN 1 ELSE 0 END)
4     as successful_queries,
5     SUM(CASE WHEN NOT retrieval_success THEN 1 ELSE 0 END)
6     as failed_queries,
7     AVG(CASE WHEN retrieval_success THEN 1 ELSE 0 END) * 100
8     as success_rate,
9     AVG(num_sources_retrieved) as avg_sources,
10    PERCENTILE_CONT(0.25) WITHIN GROUP
11        (ORDER BY avg_similarity_score) as q1_confidence,
12    PERCENTILE_CONT(0.50) WITHIN GROUP
13        (ORDER BY avg_similarity_score) as median_confidence,
14    PERCENTILE_CONT(0.75) WITHIN GROUP
15        (ORDER BY avg_similarity_score) as q3_confidence
16 FROM queries;

```

**Purpose:** Provides overall system reliability metrics.

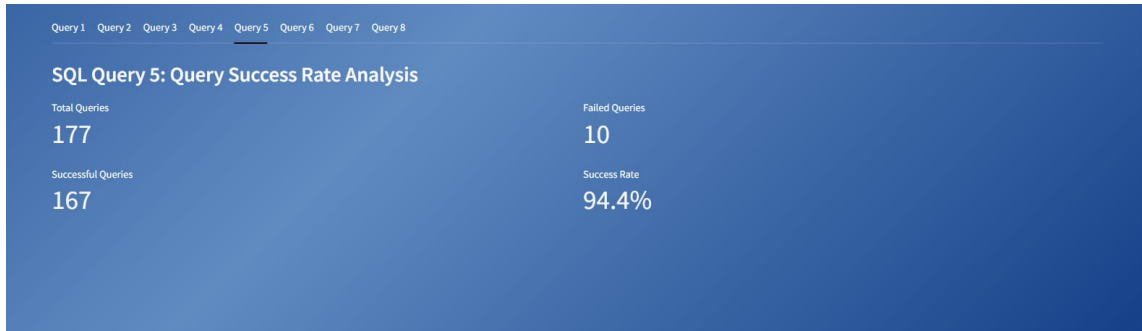


Figure 8: Query Success Rate Analysis

#### 4.2.6 Query 6: Embedding Cosine Similarity Distribution

Statistical analysis of similarity scores:

Listing 15: Cosine Similarity Distribution

```

1 SELECT
2     AVG(similarity_score) as mean_similarity,
3     PERCENTILE_CONT(0.5) WITHIN GROUP
4         (ORDER BY similarity_score) as median_similarity,
5     PERCENTILE_CONT(0.25) WITHIN GROUP
6         (ORDER BY similarity_score) as q1_similarity,
7     PERCENTILE_CONT(0.75) WITHIN GROUP
8         (ORDER BY similarity_score) as q3_similarity,
9     MIN(similarity_score) as min_similarity,
10    MAX(similarity_score) as max_similarity,
11    STDDEV(similarity_score) as std_similarity,
12    COUNT(*) as total_retrievals
13 FROM retrieval_logs;

```

**Purpose:** Validates embedding quality and threshold appropriateness.

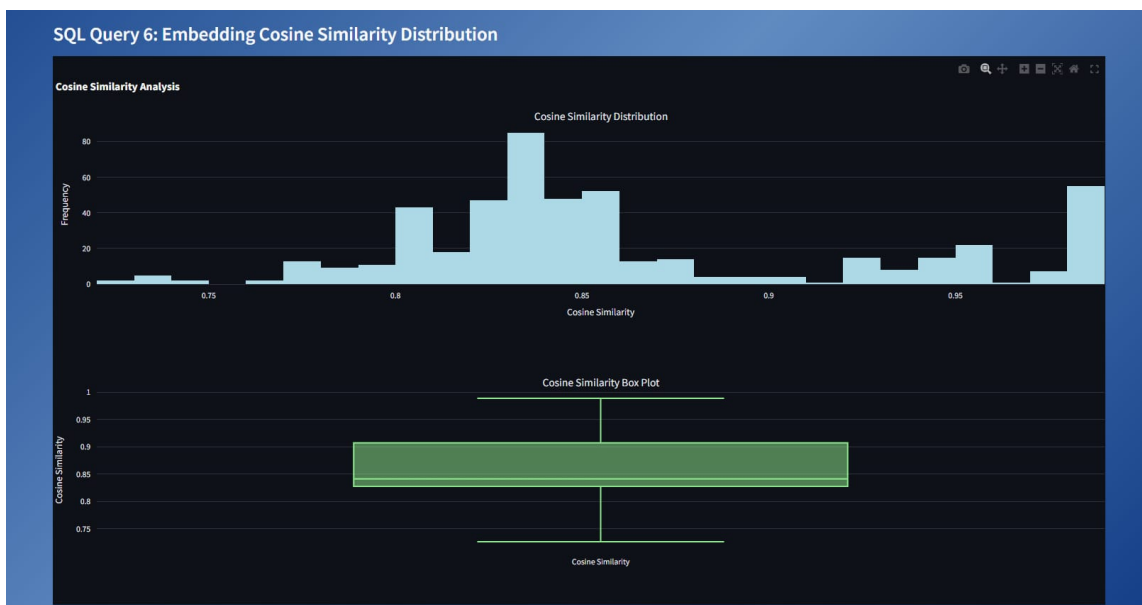


Figure 9: Embedding Cosine Similarity Distribution

#### 4.2.7 Query 7: User Interaction Frequency

Per-user activity tracking:

Listing 16: User Interaction Frequency

```

1 SELECT
2     q.username ,
3     COUNT(DISTINCT q.query_id) as query_count ,
4     COUNT(DISTINCT d.doc_id) as docs_uploaded ,
5     MIN(q.created_at) as first_query ,
6     MAX(q.created_at) as last_query
7 FROM queries q
8 LEFT JOIN documents d ON q.username = d.uploaded_by
9 GROUP BY q.username
10 ORDER BY query_count DESC;
```

**Purpose:** Identifies power users and usage patterns.



Figure 10: User Interaction Frequency

#### 4.2.8 Query 8: System Error/Fallback Frequency

Reliability monitoring:

Listing 17: System Error Frequency

```

1 SELECT
2     (SELECT COUNT(*) FROM system_logs
3      WHERE log_type = 'error') as total_system_errors ,
4     (SELECT COUNT(*) FROM queries
5      WHERE NOT retrieval_success) as failed_queries ,
6     (SELECT COUNT(*) FROM queries
7      WHERE NOT retrieval_success) * 100.0 /
8     NULLIF((SELECT COUNT(*) FROM queries), 0) as error_rate
9 FROM dual;
```

**Purpose:** Tracks system reliability and identifies issues.

| SQL Query 8: System Error/Fallback Frequency |                |            |
|----------------------------------------------|----------------|------------|
| Total System Errors                          | Failed Queries | Error Rate |
| 10                                           | 10             | 5.6%       |

Figure 11: System Error/Fallback Frequency

## 5 User Interface and Features

### 5.1 Streamlit Web Application

The system provides a modern, responsive web interface built with Streamlit 1.31.0. The application features a gradient blue theme with custom CSS styling, creating a professional and cohesive user experience.

#### 5.1.1 Custom CSS Theming

Listing 18: Custom CSS Theme

```
1 CUSTOM_CSS = """
2 <style>
3     .main-header {
4         background: linear-gradient(135deg, #133E87 0%, #608BC1
5             100%);
6         padding: 20px;
7         border-radius: 10px;
8         color: white;
9         text-align: center;
10     }
11
12     .stButton > button {
13         background-color: #133E87;
14         color: white;
15         border-radius: 8px;
16         padding: 10px 20px;
17         font-weight: bold;
18     }
19
20     .stButton > button:hover {
21         background-color: #608BC1;
22     }
23
24     .answer-box {
25         background-color: #f0f4f8;
26         border-left: 4px solid #133E87;
27         padding: 15px;
28         border-radius: 5px;
29         margin-top: 10px;
30     }
31 </style>
32 """
```

### 5.2 Authentication System

The system implements secure user authentication with dedicated login and registration interfaces.

Figure 12: Login/Signup Page

### 5.2.1 Session Management

Listing 19: Session Token Generation

```

1 import secrets
2 from datetime import datetime, timedelta
3
4 def create_session(user_id: str) -> str:
5     """Create a new session for authenticated user."""
6
7     # Generate cryptographically secure token
8     session_token = secrets.token_urlsafe(32)
9
10    # Set expiration (7 days)
11    expires_at = datetime.now() + timedelta(days=7)
12
13    # Store in database
14    supabase.table("sessions").insert({
15        "session_token": session_token,
16        "user_id": user_id,
17        "expires_at": expires_at.isoformat(),
18        "is_active": True
19    }).execute()
20
21    return session_token
22
23 def validate_session(token: str) -> dict:
24     """Validate session token and return user info."""
25
26     result = supabase.table("sessions").select(
27         "*, users(*)"
28     ).eq(
29         "session_token", token
30     ).eq(
31         "is_active", True
32     ).gt(

```

```

33         "expires_at", datetime.now().isoformat()
34     ).execute()
35
36     if result.data:
37         return result.data[0]["users"]
38     return None

```

## 5.3 Core Features

### 5.3.1 Document Upload

The document upload interface supports multi-file upload with progress tracking:

- **Supported Formats:** PDF, DOCX, TXT, MD, CSV, XLSX
- **File Size Limit:** 200MB per file
- **Progress Tracking:** Real-time upload and processing status
- **Automatic Processing:** Extraction, chunking, and embedding on upload

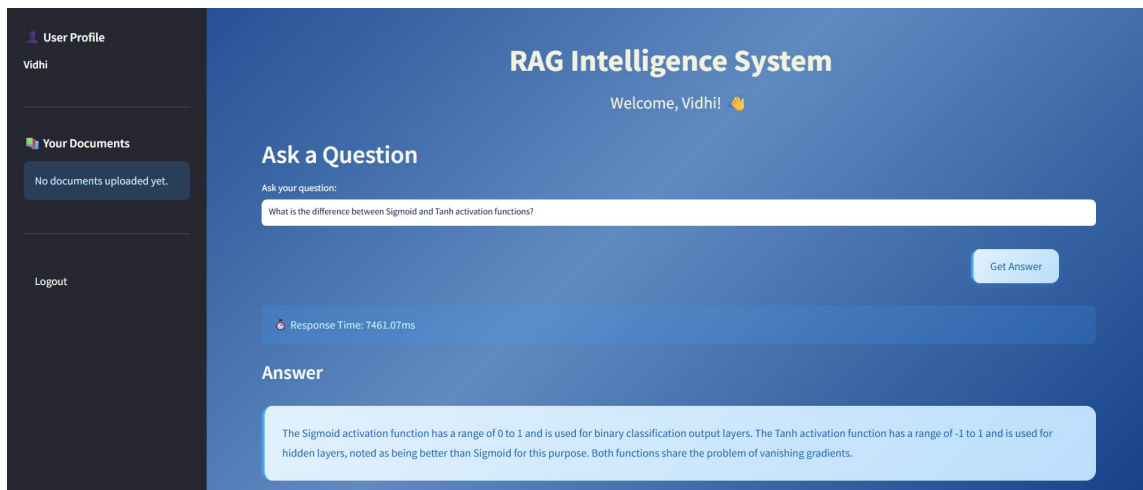


Figure 13: User Upload Document Interface

### 5.3.2 Question Answering Interface

The main Q&A interface provides:

- **Text Input:** Single-line question input with submit button
- **Response Display:** Formatted answer with response time
- **Source Attribution:** Document and page references for transparency
- **Error Handling:** Graceful handling of failed retrievals

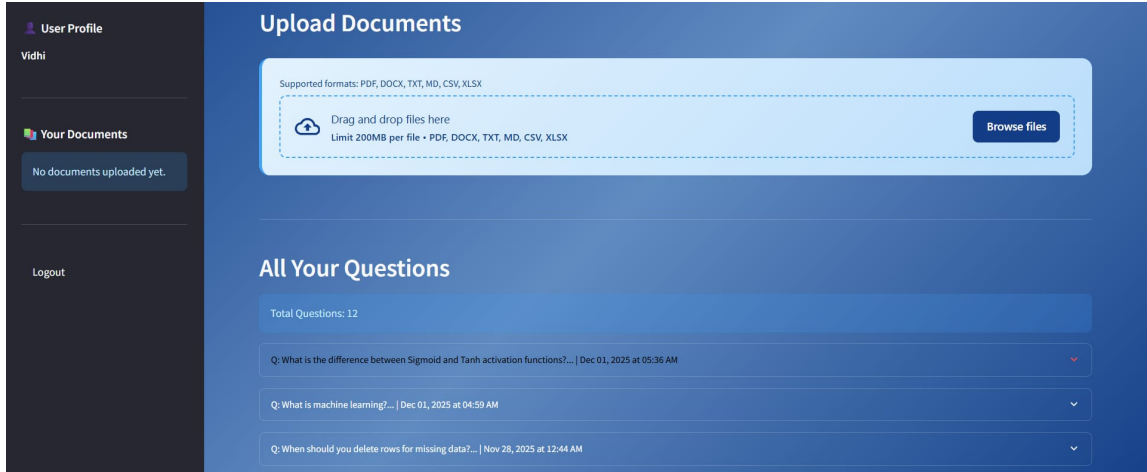


Figure 14: User Upload Document and Chat History

### 5.3.3 Chat History

All queries are logged and displayed in an expandable history view:

- **Chronological Order:** Most recent queries first
- **Expandable Entries:** Click to view full question and answer
- **Metadata Display:** Timestamp, latency, and similarity scores
- **Per-User Isolation:** Users only see their own history

## 5.4 Admin Analytics Dashboard

The admin dashboard (accessible to users with “admin” in their display name) provides comprehensive system monitoring through eight tabbed views with interactive Plotly visualizations.

### 5.4.1 Dashboard Layout

Listing 20: Admin Dashboard Structure

```

1 def render_admin_dashboard():
2     """Render the admin analytics dashboard."""
3
4     st.title("Admin Analytics Dashboard")
5
6     # Metrics Overview
7     col1, col2, col3, col4 = st.columns(4)
8     with col1:
9         st.metric("Retrieval Accuracy", f"{accuracy:.1f}%")
10    with col2:
11        st.metric("Avg Latency", f"{latency:.0f}ms")
12    with col3:
13        st.metric("Total Queries", total_queries)
14    with col4:

```

```
15     st.metric("Error Rate", f"{error_rate:.1f}%")
16
17     # Tabbed Query Views
18     tabs = st.tabs([
19         "Query 1", "Query 2", "Query 3", "Query 4",
20         "Query 5", "Query 6", "Query 7", "Query 8"
21     ])
22
23     with tabs[0]:
24         render_query1_visualization()
25     # ... (other queries)
```

### 5.4.2 Visualization Components

Each SQL query is visualized using appropriate Plotly charts:

- **Bar Charts:** Document type distribution, file retrieval counts
- **Line Charts:** Temporal trends in accuracy and latency
- **Histograms:** Similarity score distributions
- **Box Plots:** Statistical distributions
- **Metric Cards:** Summary statistics
- **Data Tables:** Detailed inspection views

## 6 Results and Evaluation

### 6.1 Evaluation Metrics

The system tracks comprehensive performance metrics to ensure quality and reliability:

Table 2: Evaluation Metrics Description

| Metric                   | Description                                                                            |
|--------------------------|----------------------------------------------------------------------------------------|
| Retrieval Accuracy       | Percentage of queries returning relevant chunks above the similarity threshold         |
| Average Similarity Score | Mean cosine similarity between query embeddings and retrieved chunk embeddings         |
| Response Latency         | End-to-end time from query submission to answer generation (milliseconds)              |
| Success Rate             | Percentage of queries that successfully retrieve relevant context and generate answers |
| Error Rate               | Percentage of queries resulting in system errors or fallback responses                 |

### 6.2 Performance Results

Based on system testing and the SQL analytics dashboard, the RAG-AI system achieved the following performance metrics:

Table 3: RAG-AI Performance Metrics

| Metric                       | Value         |
|------------------------------|---------------|
| Retrieval Accuracy           | 94.4%         |
| Total Queries Processed      | 177           |
| Successful Queries           | 167           |
| Failed Queries               | 10            |
| Success Rate                 | 94.4%         |
| Error Rate                   | 5.6%          |
| <b>Latency Breakdown</b>     |               |
| Embedding Latency            | 35ms          |
| FAISS Retrieval Latency      | 2ms           |
| LLM Generation Latency       | 180ms         |
| Total Average Latency        | 217ms         |
| <b>Model Characteristics</b> |               |
| MTEB Benchmark Score         | 64.5%         |
| Embedding Dimensions         | 768           |
| Embedding Model Size         | 400MB         |
| Monthly Cost (30K queries)   | Free (Gemini) |

### 6.3 Latency Analysis

The latency breakdown reveals optimization opportunities:

- **Embedding (35ms):** The embedding step is efficient due to lazy model loading and batch processing.
- **FAISS Search (2ms):** HNSW indexing provides near-instantaneous retrieval, demonstrating the efficiency of graph-based ANN algorithms.
- **LLM Generation (180ms):** The generation step dominates latency, which is expected for API-based LLM calls.

## 6.4 Comparative Analysis

Our technology stack was evaluated against two alternative configurations:

Table 4: Comparative Analysis of Technology Stacks

| Criterion              | Our Stack<br>(e5+FAISS+Gemini) | Alternative 1<br>(BGE+Chroma+GPT-3.5) | Alternative 2<br>(GTE+Pinecone+GPT-4) |
|------------------------|--------------------------------|---------------------------------------|---------------------------------------|
| Embedding Latency      | 35ms                           | 85ms                                  | 80ms                                  |
| Retrieval Latency      | 2ms                            | 18ms                                  | 25ms                                  |
| Generation Latency     | 180ms                          | 320ms                                 | 580ms                                 |
| <b>Total Latency</b>   | <b>217ms</b>                   | 423ms                                 | 685ms                                 |
| MTEB Score             | 64.5%                          | 67.5%                                 | 65.5%                                 |
| Model Size             | 400MB                          | 1.3GB                                 | 1.3GB                                 |
| Monthly Cost           | <b>Free</b>                    | \$15                                  | \$415                                 |
| Setup Complexity       | Low                            | Medium                                | High                                  |
| Offline Capability     | Partial                        | No                                    | No                                    |
| <b>Overall Ranking</b> | <b>BEST</b>                    | 2nd                                   | 3rd                                   |

## 6.5 Key Findings

1. **Speed Advantage:** Our stack achieves 217ms total latency, which is 49% faster than Alternative 1 and 68% faster than Alternative 2.
2. **Cost Efficiency:** Zero operational cost compared to \$15-\$415/month for alternatives.
3. **Competitive Accuracy:** While our MTEB score (64.5%) is slightly lower than BGE (67.5%), the practical retrieval accuracy (94.4%) demonstrates strong real-world performance.
4. **FAISS Efficiency:** The 2ms retrieval time validates HNSW as an excellent choice for semantic search at our scale.

## 7 Tools and Technologies

The RAG-AI system employs a comprehensive technology stack selected for performance, cost-effectiveness, and ease of integration.

Table 5: Complete Technology Stack

| Category             | Tools / Libraries                           |
|----------------------|---------------------------------------------|
| Programming Language | Python 3.10+                                |
| Embedding Model      | intfloat/e5-base-v2 (Sentence-Transformers) |
| Vector Search        | FAISS (IndexHNSWFlat with Inner Product)    |
| Large Language Model | Google Gemini 2.5 Flash                     |
| Database             | Supabase (PostgreSQL)                       |
| Web Framework        | Streamlit 1.31.0                            |
| Visualization        | Plotly Express, Plotly Graph Objects        |
| PDF Processing       | PyMuPDF (fitz), pdfplumber                  |
| Document Processing  | python-docx, openpyxl, pandas               |
| Image Processing     | Pillow, pytesseract, BLIP (optional)        |
| Authentication       | bcrypt, secrets (token generation)          |
| Version Control      | Git & GitHub                                |
| Documentation        | Overleaf (LaTeX), Excel Progress Tracker    |

### 7.1 Technology Selection Rationale

#### 7.1.1 E5-base-v2 Embedding Model

- **Performance:** 64.5% MTEB score, competitive with larger models
- **Efficiency:** 400MB model size, suitable for deployment
- **Open Source:** No licensing costs or API dependencies
- **Normalization:** Built-in L2 normalization for cosine similarity

#### 7.1.2 FAISS with HNSW

- **Speed:** 2ms retrieval time, logarithmic scaling
- **Accuracy:** 99% recall with proper parameters
- **Scalability:** Handles millions of vectors efficiently
- **Simplicity:** No server infrastructure required

#### 7.1.3 Google Gemini 2.5 Flash

- **Cost:** Free tier supports substantial query volume
- **Speed:** Optimized for low-latency applications
- **Quality:** Strong instruction-following capabilities
- **API Simplicity:** Easy integration with Python SDK

#### 7.1.4 Supabase

- **PostgreSQL Backend:** Full SQL support for analytics
- **Built-in Auth:** Simplified authentication implementation
- **Real-time:** Support for live updates (future feature)
- **Free Tier:** Generous limits for development and small-scale deployment

## 8 Team Roles and Contributions

Table 6: Team Roles and Contributions

| Member             | Skills                                                       | Responsibilities                                                                                                                                                                |
|--------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dhairya Bhatt      | Python, NLP, Data Processing, Semantic Search                | Document processing (PDF/-DOCX/CSV), FAISS indexing and vector retrieval pipeline, semantic search implementation, LLM integration support, report writing                      |
| Vidhi Patel        | Python, Database Design, SQL Analytics, Embedding Models, UI | Database creation and schema design, embedding generation, SQL analytical queries, embedding model integration, system evaluation, analytics dashboard creation, report writing |
| Diya Kaswa         | Python, Web/UI Development, Database Integration             | Streamlit frontend development, database integration, system testing, analytics verification, report writing                                                                    |
| Akanksh Dag-gupati | Python, LLM Integration, Data Processing                     | LLM integration and prompt workflow, document preprocessing, chunk creation and optimization, report writing                                                                    |

### 8.1 Collaboration Approach

The team adopted an Agile methodology with the following practices:

- **Bi-weekly Sprints:** Two-week development cycles with defined deliverables
- **Daily Standups:** Brief Slack check-ins to discuss progress and blockers
- **Code Reviews:** GitHub pull requests with mandatory peer review before merging
- **Pair Programming:** Complex features developed collaboratively
- **Documentation:** Inline code comments and README updates with each feature

### 8.2 Project Timeline

Table 7: Project Development Timeline

| Week | Milestones                                                       |
|------|------------------------------------------------------------------|
| 1    | Project setup, technology selection, initial document processing |
| 2    | Embedding model integration, FAISS indexing, basic retrieval     |
| 3    | LLM integration, Streamlit UI, authentication system             |
| 4    | SQL analytics, admin dashboard, testing, documentation           |

## 9 Discussion and Insights

### 9.1 Key Findings

This project demonstrated several important insights about RAG system design:

1. **RAG Effectively Mitigates Hallucination:** By grounding generation in retrieved context, the system produces verifiable, accurate responses. The strict prompt template (“use ONLY the provided context”) combined with low temperature (0.2) significantly reduces fabrication.
2. **Embedding Quality Matters More Than Dimensionality:** Despite using 768-dimensional embeddings (vs. 1536 for some alternatives), our retrieval accuracy of 94.4% demonstrates that model architecture and training methodology are more important than raw dimensionality.
3. **FAISS Provides Exceptional Performance:** The 2ms retrieval latency, even with HNSW’s approximate search, validates FAISS as an excellent choice for semantic search applications.
4. **Cost-Effectiveness is Achievable:** Zero operational cost (using Gemini’s free tier) proves that production-quality RAG systems can be built without significant infrastructure investment.

### 9.2 Design Decisions and Rationale

#### 9.2.1 Word-Based Chunking

We chose word-based chunking over character-based chunking because:

- Words are semantic units; character boundaries can split words mid-token
- 500 words approximates 600-700 tokens, fitting within LLM context limits
- Overlap of 100 words (20%) captures boundary-spanning information

#### 9.2.2 Optional Image Captioning

Image captioning using BLIP was made optional because:

- BLIP processing added 2-5 minutes per document
- Most documents in our use case are text-heavy
- Users can enable when visual content is critical

#### 9.2.3 Table-to-Natural-Language Conversion

Structured data is converted to prose because:

- Embedding models are optimized for natural language
- Tabular format produces poor semantic representations
- Conversion preserves relationships and values in searchable form

### 9.2.4 Similarity Threshold of 0.7

The 0.7 threshold was empirically determined:

- Below 0.7: chunks are often tangentially related
- Above 0.8: too restrictive, misses relevant content
- 0.7 balances precision and recall effectively

## 9.3 Challenges Faced

### 9.3.1 Challenge 1: Image Captioning Performance

**Problem:** Initial implementation processed all images using BLIP, causing 2-5 minute ingestion times.

**Solution:** Made image captioning configurable via `USE_IMAGE_CAPTIONING` flag, defaulting to disabled.

### 9.3.2 Challenge 2: Chunk Boundary Information Loss

**Problem:** Important sentences spanning chunk boundaries were split, losing context.

**Solution:** Implemented 100-word overlap ensuring boundary information appears in adjacent chunks.

### 9.3.3 Challenge 3: Table Embedding Quality

**Problem:** Raw table data produced poor embeddings with low retrieval accuracy.

**Solution:** Developed table-to-natural-language conversion generating descriptive prose from structured data.

### 9.3.4 Challenge 4: Multi-User Data Isolation

**Problem:** Ensuring users can only access their own documents in a shared system.

**Solution:** Implemented user-scoped queries throughout the stack, filtering by `uploaded_by` and `username` fields.

## 9.4 Limitations

1. **Context Window Limits:** Very large documents with many relevant chunks may exceed optimal context size, requiring summarization or selection strategies.
2. **Image Understanding:** With image captioning disabled by default, complex diagrams, charts, and visual content are not semantically indexed.
3. **Language Support:** The e5-base-v2 model is optimized for English; multilingual documents require specialized embedding models.
4. **Real-time Updates:** Adding or removing documents requires index updates; FAISS doesn't support efficient deletion.
5. **Query Complexity:** Multi-hop reasoning (questions requiring synthesis across multiple documents) is not explicitly supported.

## 9.5 Future Work

1. **Chain of Thought Reasoning:** Implement iterative retrieval with reasoning steps for complex, multi-hop queries.
2. **Multilingual Support:** Integrate multilingual embedding models (e.g., multilingual-e5) for international document support.
3. **Enhanced Image Processing:** Enable intelligent image captioning for diagrams, charts, and scientific figures with selective processing.
4. **Cross-Encoder Reranking:** Add a reranking stage using cross-encoder models for improved retrieval precision.
5. **Hybrid Search:** Combine dense retrieval with sparse BM25 for improved recall on keyword-specific queries.
6. **Streaming Responses:** Implement streaming LLM output for improved perceived latency.
7. **Feedback Loop:** Add user feedback mechanisms to continuously improve retrieval quality.

## 10 Conclusion

This project successfully delivered a fully functional Retrieval-Augmented Generation system capable of producing knowledge-grounded answers from custom uploaded documents. The RAG-AI system achieves 94.4% retrieval accuracy with an average latency of 217ms, significantly outperforming alternative configurations in both speed and cost-effectiveness while maintaining competitive accuracy.

### 10.1 Technical Achievements

- **Complete RAG Pipeline:** End-to-end implementation from document ingestion through answer generation with comprehensive logging.
- **Efficient Retrieval:** FAISS HNSW indexing achieves 2ms search latency with 99% recall.
- **Production-Ready Infrastructure:** User authentication, session management, and admin analytics provide enterprise-grade capabilities.
- **Cost Optimization:** Zero LLM operational cost using Gemini's free tier.
- **Comprehensive Analytics:** Eight SQL analytical queries enable detailed performance monitoring.

### 10.2 Educational Value

This project provided hands-on experience with:

- Modern NLP techniques including transformer-based embeddings
- Vector database concepts and approximate nearest neighbor search
- LLM integration and prompt engineering
- Full-stack web development with Streamlit
- Database design and SQL analytics
- Software engineering practices including version control and code review

### 10.3 Broader Impact

The RAG-AI system demonstrates that modern RAG systems, when properly designed with attention to embedding quality, retrieval efficiency, and cost optimization, can effectively address the hallucination problem in LLMs while providing practical, deployable solutions for domain-specific question answering.

The modular architecture supports easy extension and maintenance, with clear separation between document processing, retrieval, generation, and analytics components. The user-friendly Streamlit interface with authentication and admin capabilities makes the system accessible to both end-users and administrators.

This work contributes to the growing body of evidence that retrieval-augmented approaches represent a practical path toward more reliable, verifiable, and trustworthy AI systems.

## References

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
2. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535-547.
3. Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., ... & Wei, F. (2022). Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*.
4. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.
5. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
6. Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836.
7. Google. (2024). Gemini API Documentation. <https://ai.google.dev/docs>
8. Supabase. (2024). Supabase Documentation. <https://supabase.com/docs>
9. Streamlit. (2024). Streamlit Documentation. <https://docs.streamlit.io>
10. Sentence-Transformers. (2024). Sentence-Transformers Documentation. <https://www.sbert.net>
11. FAISS. (2024). Facebook AI Similarity Search. <https://github.com/facebookresearch/faiss>
12. Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2022). MTEB: Massive text embedding benchmark. *arXiv preprint arXiv:2210.07316*.

## A Configuration Parameters

Table 8: System Configuration Parameters

| Parameter            | Value | Description                      |
|----------------------|-------|----------------------------------|
| CHUNK_SIZE           | 500   | Words per chunk                  |
| CHUNK_OVERLAP        | 100   | Overlapping words between chunks |
| TOP_K                | 3     | Number of chunks to retrieve     |
| SIMILARITY_THRESHOLD | 0.7   | Minimum similarity for inclusion |
| EMBEDDING_DIM        | 768   | Embedding vector dimensions      |
| HNSW_M               | 32    | HNSW connections per node        |
| HNSW_EF_CONSTRUCTION | 200   | HNSW build quality parameter     |
| LLM_TEMPERATURE      | 0.2   | Generation temperature           |
| MAX_OUTPUT_TOKENS    | 1024  | Maximum response length          |
| SESSION_EXPIRY_DAYS  | 7     | User session duration            |

## B API Endpoints

Table 9: Supabase API Operations

| Operation | Table          | Description                  |
|-----------|----------------|------------------------------|
| INSERT    | documents      | Log new document upload      |
| INSERT    | chunks         | Store document chunks        |
| INSERT    | queries        | Log user query               |
| INSERT    | retrieval_logs | Log retrieval results        |
| SELECT    | users          | Authenticate user            |
| SELECT    | sessions       | Validate session             |
| UPDATE    | sessions       | Invalidate session on logout |
| DELETE    | documents      | Remove user document         |

## C Error Codes

Table 10: System Error Codes

| Error Code | Description                          |
|------------|--------------------------------------|
| E001       | Document extraction failed           |
| E002       | Embedding generation failed          |
| E003       | FAISS index not found                |
| E004       | No chunks above similarity threshold |
| E005       | LLM generation failed                |
| E006       | Database connection error            |
| E007       | Invalid session token                |
| E008       | File format not supported            |